LONDON METROPOLITAN UNIVERSITY



Islington college
(इस्लिङ्टन कलेज)

CS5002NI Software Engineering

35% Weightage Coursework (Final Submission)

AY 2025- 26

Credit: 30

Student Name: Sakshyam Prabhat Adhikari

London Met ID: 24046677

College ID: NP01AI4A240141

Assignment Due Date: April 30, 2026 (6:00pm)

Assignment Submission Date: April 30, 2026

Word Count: 5176

I confirm that I understand my coursework needs to be submitted online via Google Classroom under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

Table of Contents

1. WBS and Gantt Chart **1-4**

1.1 WBS 1-2

1.2 Gantt Chart 2-4

2. Use Case Model **5-13**

2.1 Use Case Diagram 5-6

2.2 High Level Use Case Descriptions 7-10

2.3 Expanded Use Case Descriptions 11-13

3. Collaboration Diagram **13-14**

4. Activity Diagram **14-15**

5. Table and Class Diagram **16-20**

5.1 Table 16-17

5.2 Class Diagram 18-20

6. Further Planning **20-22**

6.1 Choice of Development Methodology 20

6.2 Architectural Choice 21

6.3 Design Pattern 21

6.4 Development Plan 21

6.5 Testing Plan 22

6.6 Maintenance Plan 22

6.7 Conclusion 22

7. Design Prototypes **22-38**

Table of Figures

1. Fig 1: Work Breakdown Structure (WBS) 2
2. Fig 2: Gantt Chart Part 1 4
3. Fig 3: Gantt Chart Part 2 4
4. Fig 4: Use Case Model Diagram 6
5. Fig 5: Collaboration Diagram 14
6. Fig 6: Activity Diagram 15
7. Fig 7: Analysis Class Diagram 20
8. Fig 8: Prototype 1: Home Page 23
9. Fig 9: Prototype 2: Login Page 24
10. Fig 10: Prototype 3: Registration Page 25
11. Fig 11: Prototype 4: Reset Password Page 26
12. Fig 12: Prototype 5: User Dashboard 27
13. Fig 13: Prototype 6: Check Table Availability 28
14. Fig 14: Prototype 7: Book Table Page 29
15. Fig 15: Prototype 8: Beverage Selection Page 30
16. Fig 16: Prototype 9: Advance Payment Page 31
17. Fig 17: Prototype 10: Reservation Confirmation Page 32
18. Fig 18: Prototype 11: Cancel Reservation Page 33
19. Fig 19: Prototype 12: Order Placement Page 34
20. Fig 20: Prototype 13: Delivery Option Page 35
21. Fig 21: Prototype 14: Bill & Receipt Page 36
22. Fig 22: Prototype 15: Admin Dashboard Page 37
23. Fig 23: Prototype 16: Rate Experience/Feeback Page 38

Table of Tables

1. Table 1: Expanded Use Case 2 – Book Table (Reserve)..... 11
2. Table 2: Expanded Use Case 6 – Place Food/Beverage Order 12-13
3. Table 3: Table showing all use cases and domain classes 16-17

1. WBS and Gantt Chart

1.1 WBS

The Work Breakdown Structure (WBS) for the Gokyo Bistro Management System presents a hierarchical decomposition of the entire project into smaller, manageable components. The project is divided into five major phases: Initiation, Planning, Execution, Control, and Closure.

The **Initiation phase** focuses on identifying project goals, understanding the problems faced by the restaurant, defining system objectives, developing the project charter, and conducting feasibility studies. It also includes setting up the Agile environment and defining roles and tools required for project execution.

The **Planning phase** involves detailed requirement analysis for core functionalities such as login, table booking, payment, and admin features. It also includes sprint planning, backlog creation, system design planning (including architecture, database, and UI/UX), as well as risk and resource management.

The **Execution phase** is organized into four Agile sprints, each targeting specific system functionalities: Authentication System, Table Booking System, Payment and Order System, and Admin and Feedback System. Each sprint follows a structured process including sprint planning, implementation, testing, deployment, and review, ensuring iterative development and continuous improvement.

The **Control phase** runs alongside execution and focuses on monitoring progress, managing risks, ensuring quality through validation and code checks, and handling any requirement changes.

Finally, the **Closure phase** includes delivering the final system, preparing technical documentation and user manuals, conducting final testing, evaluating project success, and submitting the final report.

Overall, the WBS ensures a systematic and organized approach to project execution by breaking down complex tasks into manageable units aligned with Agile Scrum practices.

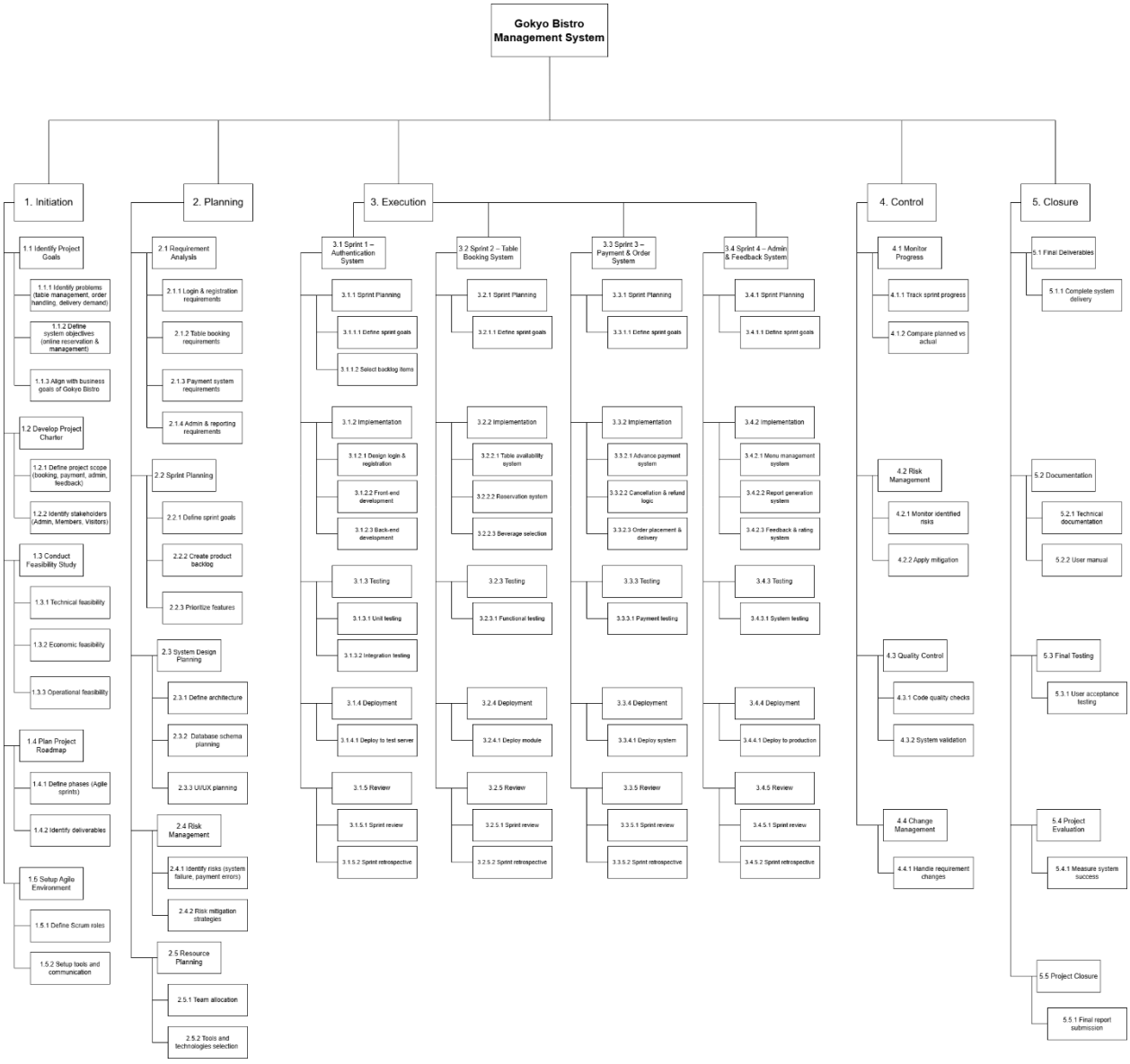


Fig 1: Work Breakdown Structure (WBS)

1.2 Gantt Chart

The Gantt chart for the Gokyo Bistro Management System has been **developed by using an online software called TeamGantt** and is **based on the Work Breakdown Structure (WBS)** and **follows the Agile Scrum methodology.** The project is planned over a period of approximately six weeks, starting from **18 March 2026 to 27 April 2026**.

The project is divided **into five major phases: Initiation, Planning, Execution, Control, and Closure.**

The Initiation and Planning phases involve identifying project goals, developing the project charter, conducting feasibility analysis, requirement gathering, sprint planning, resource planning, and system design planning. These phases are mostly sequential to ensure that the project has a clear foundation before sprint execution begins.

The Execution phase is organized into four Agile sprints. Each sprint focuses on a specific set of system features: Sprint 1 focuses on the authentication system, Sprint 2 focuses on the table booking system, Sprint 3 focuses on the payment and order system, and Sprint 4 focuses on the admin and feedback system. Tasks within each sprint are scheduled to overlap, reflecting the Agile approach where implementation, testing, deployment, and review activities occur iteratively and concurrently.

The Control phase runs throughout the sprint execution period and includes progress monitoring, risk management, quality control, and change management. This ensures that the project remains on track and any risks or changes are handled during development.

Finally, the Closure phase includes final deliverables, documentation, final testing, project evaluation, project closure, and final report submission.

Dependencies are applied only between major milestones and phases to maintain logical flow without restricting Agile flexibility. Overall, the Gantt chart provides a clear and structured timeline for completing the project efficiently using Agile Scrum principles within the coursework period.

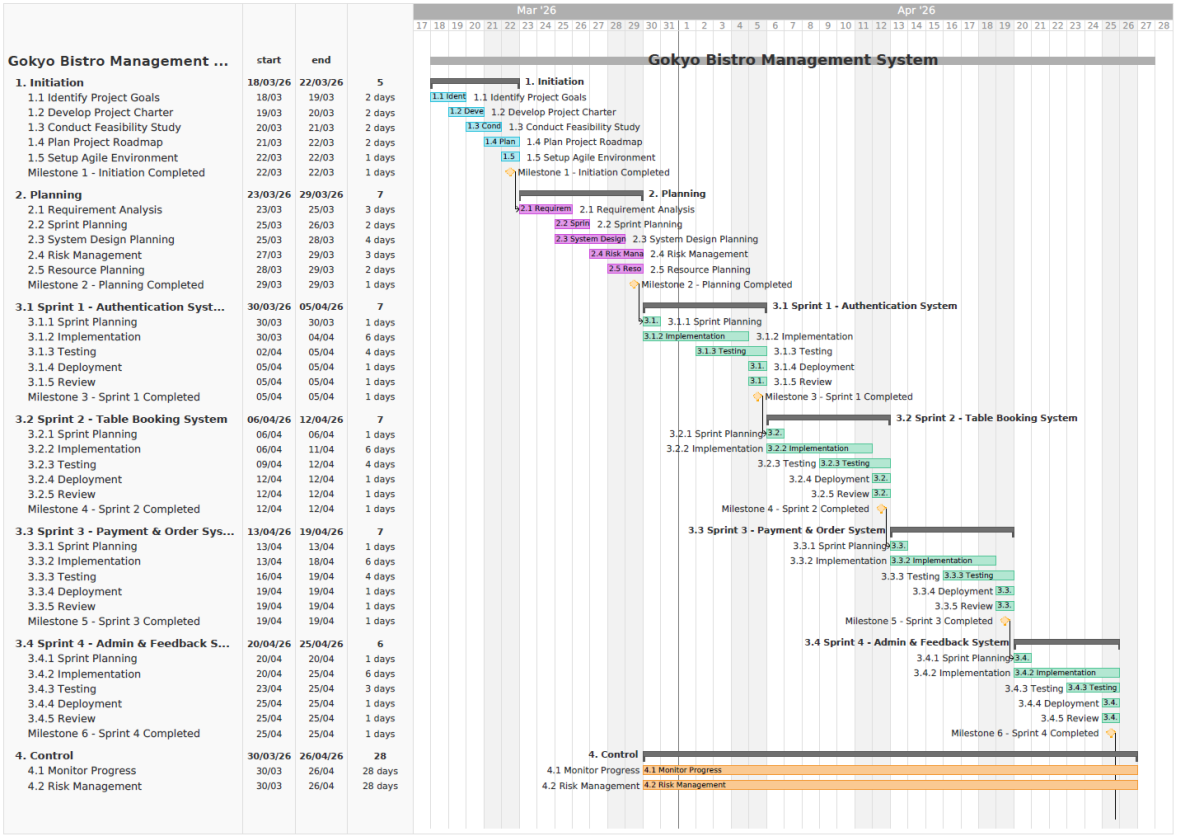


Fig 2: Gantt Chart Part 1

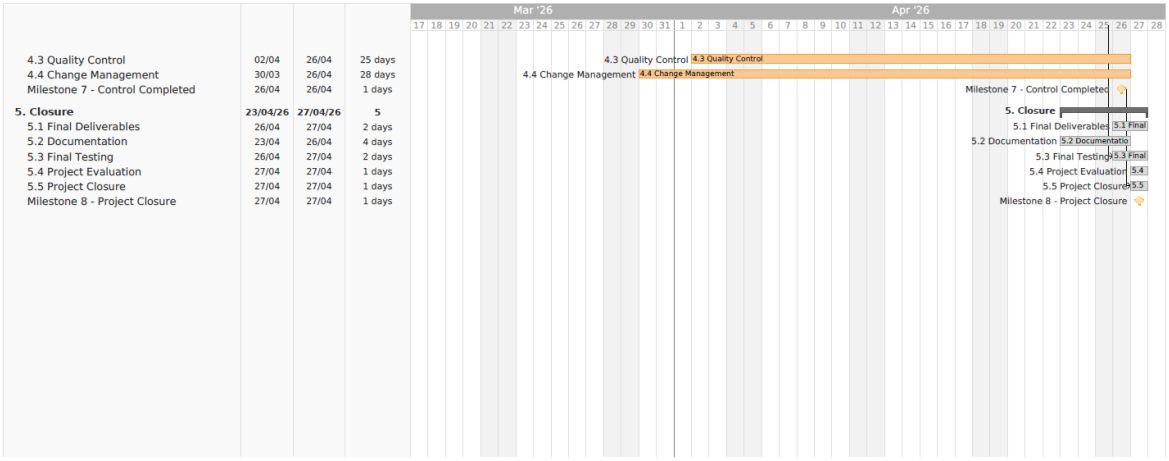


Fig 3: Gantt Chart Part 2

2. Use Case Model

2.1 Use Case Diagram

The use case diagram represents the main functionalities of the Gokyo Bistro Management System and shows how different users interact with the system. The system has three main actors: **Visitor**, **Member**, and **Admin**.

The **Visitor** can use the system to register/login, book a table, check table availability, set beverage choices, make an advance payment, cancel reservations, and rate their experience. The **Member** has similar facilities but also has additional privileges such as reserving premium wine or whiskey, placing food and beverage orders, selecting home delivery, and receiving member notifications.

The **Admin** is responsible for managing the internal operations of the system. Admin can generate bills and receipts, update the menu by adding, editing, or deleting menu items, generate financial reports, view revenue prediction and menu recommendations, and post offers or discount schemes.

The diagram also uses **<<include>>** and **<<extend>>** relationships. The **<<include>>** relationship is used where one use case is compulsory for another, such as login including authentication, booking including table availability checking, and payment including online payment. The **<<extend>>** relationship is used for optional or conditional behaviour, such as reset password extending login, premium wine/whiskey reservation extending beverage choice, cancellation fee extending cancellation, home delivery extending order placement, and revenue prediction extending report generation.

Overall, this use case diagram clearly shows the boundary of the proposed system, the roles of each actor, and the major functions required for table reservation, payment, ordering, feedback, menu management, billing, reporting, and promotional activities.

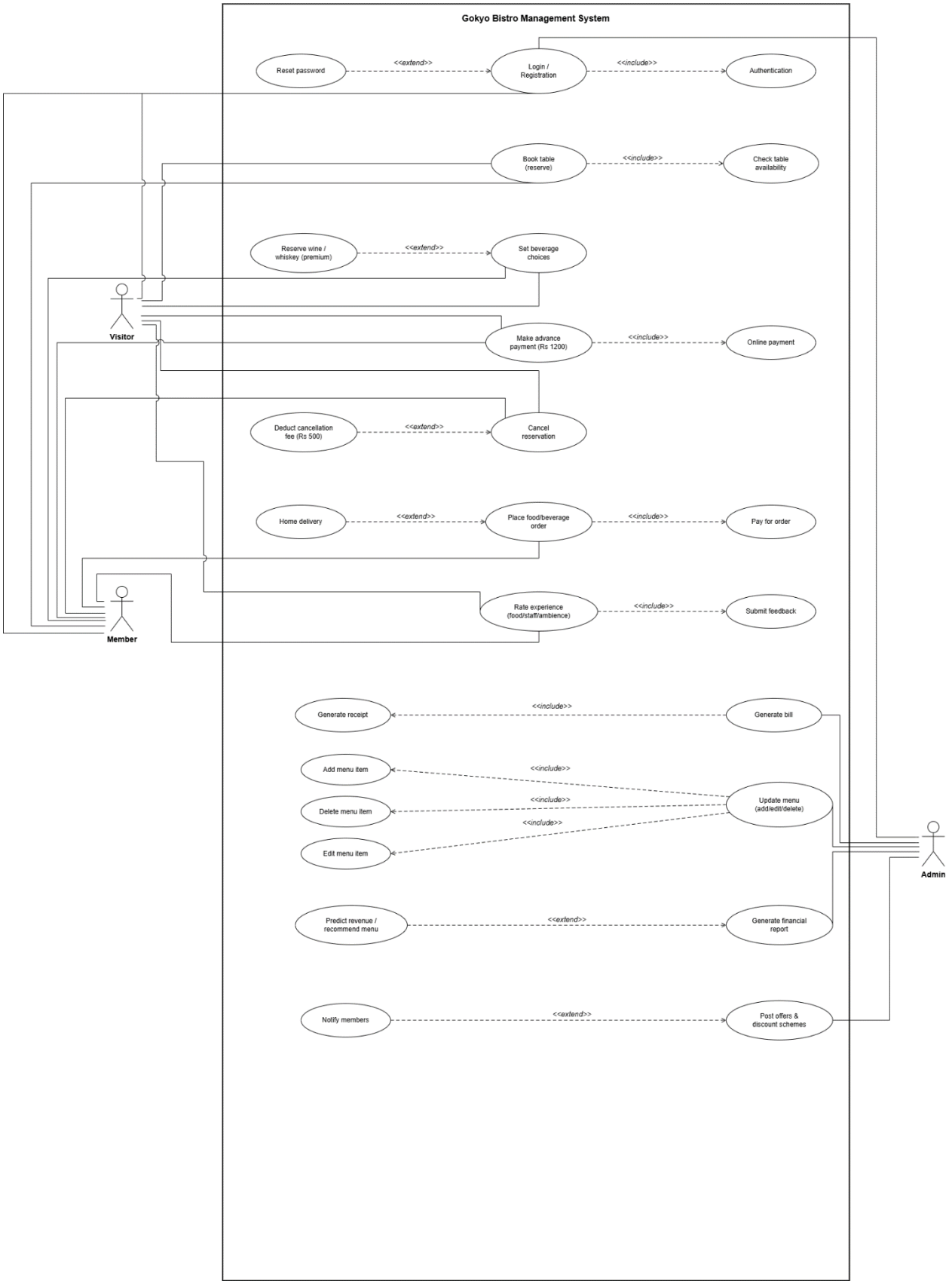


Fig 4: Use Case Model Diagram

2.2 High-Level Use Case Descriptions

The following high-level use case descriptions summarise all use cases identified in the Gokyo Bistro Management System use case diagram.

UC1 – Login/Registration

Use Case: Login / Registration

Actors: Visitor, Member, Admin (initiator)

Description: A user provides their credentials to access the system. New visitors can register an account. Registered members and admins can log in with existing credentials. The system validates credentials and grants appropriate access. The system also supports password reset as an extend relationship.

UC2 – Book Table (Reserve)

Use Case: Book Table (Reserve)

Actors: Visitor, Member (initiator)

Description: A visitor or member checks table availability for a desired date and time and makes a reservation. The system includes a check on table availability before confirming the booking. An advance payment of Rs 1200 is required to finalise the reservation.

UC3 – Set Beverage Choices

Use Case: Set Beverage Choices

Actors: Visitor, Member (initiator)

Description: While making a booking, a visitor or member can optionally select expected beverage preferences from the available menu list. Premium members have the additional option (extend relationship) to reserve specific wine or whiskey choices in advance.

UC4 – Make Advance Payment

Use Case: Make Advance Payment (Rs 1200)

Actors: Visitor, Member (initiator)

Description: To confirm a table reservation, the user is required to pay an advance of Rs 1200. Payment is processed online through an included online payment sub-use case. The system verifies payment success and confirms the reservation.

UC5 – Cancel Reservation

Use Case: Cancel Reservation

Actors: Visitor, Member (initiator)

Description: A visitor or member can cancel an existing reservation before the reserved date. If the cancellation is made on the same day as the reservation, the system extends with a deduction of Rs 500 from the advance before refunding the remainder. Cancellations made prior to the final day receive a full refund.

UC6 – Place Food/Beverage Order

Use Case: Place Food/Beverage Order

Actors: Member (initiator)

Description: A registered member can place orders for food and beverages. The use case includes an online payment step. Members can optionally extend the use case by selecting home delivery. The system confirms the order and provides an order summary.

UC7 – Rate Experience

Use Case: Rate Experience (Food / Staff / Ambience)

Actors: Visitor, Member (initiator)

Description: After a dining experience, a visitor or member can rate the food quality, staff behaviour, and overall ambience. Providing written feedback is included as part of the use case. Ratings and feedback are stored by the system for admin review.

UC8 – Generate Bill

Use Case: Generate Bill

Actors: Admin (initiator)

Description: The admin generates a bill for a completed dining session or order. The system calculates total charges based on items consumed and applies any applicable discounts. A receipt is generated as an included use case and provided to the customer.

UC9 – Update Menu

Use Case: Update Menu (Add / Edit / Delete)

Actors: Admin (initiator)

Description: The admin can manage the restaurant menu by adding new items, editing existing item details (name, description, price, category), or deleting discontinued items. These are included as sub-use cases. The updated menu is immediately reflected across the system for visitors and members.

UC10 – Generate Financial Report

Use Case: Generate Financial Report

Actors: Admin (initiator)

Description: The admin generates financial reports covering revenue, orders, and reservation data for a selected period. The system can extend this use case with an advanced revenue prediction mechanism that also recommends menu items based on historical performance data.

UC11 – Post Offers & Discount Schemes

Use Case: Post Offers & Discount Schemes

Actors: Admin (initiator)

Description: The admin posts special offers, seasonal promotions, and discount schemes on the system. The system extends this use case by notifying registered members about active offers. Offers are visible to all visitors and members when browsing the system.

2.3 Expanded Use Case Descriptions

The following two use cases are presented with expanded level descriptions, detailing typical courses of events and alternative courses.

1. Expanded UC2 – Book Table (Reserve)

Actors: Visitor / Member (initiator)

Use Case: Book Table (Reserve)

Description: A visitor or member provides booking details (date, time, number of guests, and seating preference) to the system. The system checks table availability, processes the advance payment of Rs 1200, and issues a booking confirmation.

Typical Course of Events:

Actor Action	System Response
1. The Visitor/Member navigates to the booking section and enters the desired date, time, and number of guests.	2. The system checks table availability for the specified date and time.
3. If a table is available, the Visitor/Member selects a seating preference (indoor/outdoor) and confirms the booking.	4. The system displays the booking summary and prompts for an advance payment of Rs 1200.
5. The Visitor/Member selects a payment method and provides payment details.	6. The system processes the online payment and verifies its success.
7. If payment is successful, the Visitor/Member optionally sets beverage preferences from the available list.	8. The system saves the beverage preferences and generates a unique booking confirmation ID.
9. The Visitor/Member receives the booking confirmation with all details.	10. The system marks the selected table as reserved for the specified date and time and updates availability.

Table 1: Expanded Use Case 2 – Book Table (Reserve)

Alternative Courses:

Line 2: If no table is available for the selected date/time, the system suggests alternative available slots. The Visitor/Member can either select an alternative slot or cancel the booking request.

Line 6: If the payment fails, the system notifies the Visitor/Member and prompts them to retry or use a different payment method. If payment is not completed, the reservation is not confirmed and the booking request is discarded.

Line 5: If the Visitor/Member does not have a valid payment method, the use case ends and the reservation is not saved.

2. Expanded UC6 – Place Food/Beverage Order

Actors: Member (initiator)

Use Case: Place Food/Beverage Order

Description: A registered member browses the current menu, selects food and beverage items, and places an order. The system computes the total bill and processes payment. The member may optionally choose home delivery. The system confirms the order and provides an order reference.

Typical Course of Events:

Actor Action	System Response
1. The Member logs in and navigates to the order placement section.	2. The system displays the current food and beverage menu with prices and availability.
3. The Member selects one or more food/beverage items and adds them to the order.	4. The system calculates and displays the total order amount.
5. The Member chooses between dine-in or home delivery. If home delivery is selected, the Member provides the delivery address.	6. The system confirms the delivery option and displays the final total (including any delivery charges if applicable).
7. The Member selects an online payment method and provides payment details.	8. The system processes the payment and verifies its success.
9. If payment is successful, the Member receives an order confirmation with a unique order ID.	10. The system records the order, updates inventory status, and notifies kitchen staff (or

	dispatches delivery if applicable).

Table 2: Expanded Use Case 6 – Place Food/Beverage Order

Alternative Courses:

Line 2: If a selected menu item is out of stock, the system displays it as unavailable. The Member may remove it from the order or choose an alternative item.

Line 8: If the payment fails, the system notifies the Member and prompts a retry or selection of an alternative payment method. The order is held pending and not dispatched until payment is confirmed.

Line 5: If the Member selects home delivery but provides an address outside the delivery zone, the system notifies the Member that delivery is unavailable for that location. The Member may switch to dine-in or cancel the order.

3. Collaboration Diagram

This collaboration diagram represents the **Book Table (Reserve)** use case for the Gokyo Bistro Management System. It shows how a **Visitor/Member** interacts with the system to reserve a table.

The actor first enters booking details through the **Book Table UI**. The UI sends the details to the **Make Booking** control object, which checks table availability through the **Table** object. If a table is available, the user selects the table and beverage choices, after which the system requests payment. The **Payment** object processes the payment and returns the payment status.

After successful payment, the **Make Booking** object creates the booking record, updates table availability, saves beverage choices, and generates a booking confirmation. Finally, the booking confirmation is displayed back to the Visitor/Member.

Overall, the diagram correctly shows the collaboration between boundary, control, and entity objects involved in the table booking process, including booking records, table availability, payment processing, beverage choice, and booking confirmation.

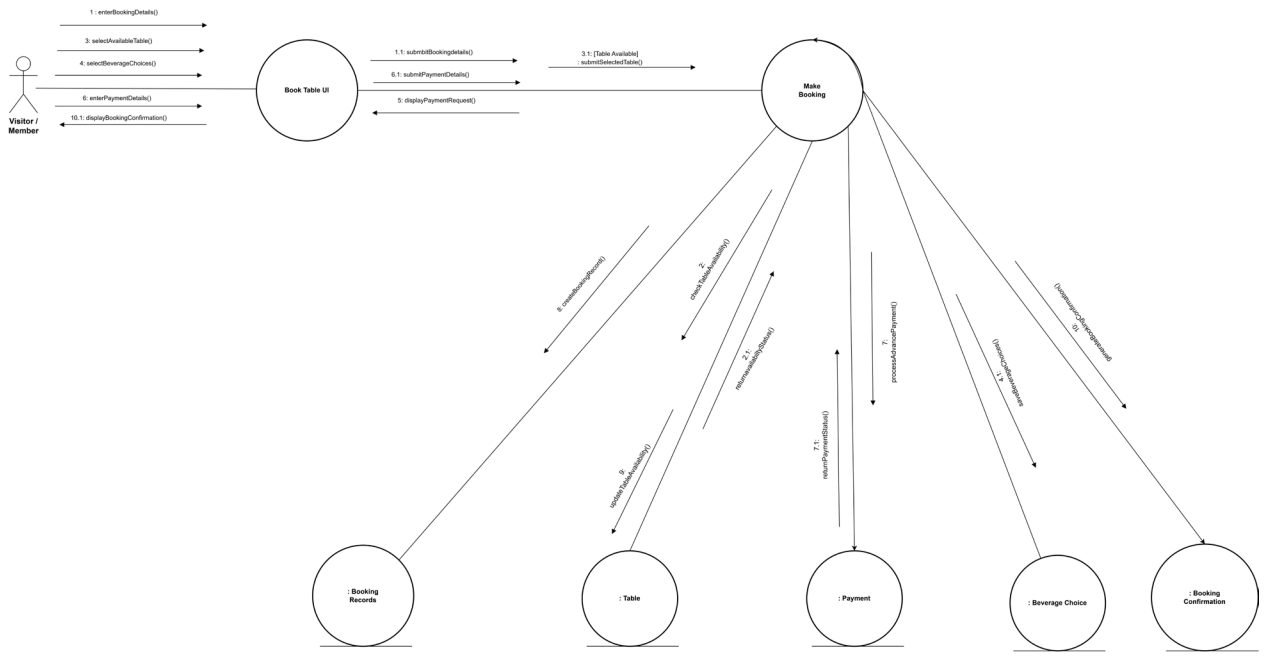


Fig 5: Collaboration Diagram

4. Activity Diagram

This activity diagram represents the **Place Food/Beverage Order (UC6)** process in the Gokyo Bistro Management System.

The process starts when the user opens the order form and browses the menu. The user then selects food and beverage items, and the system calculates the total amount. A decision is made on whether the user wants home delivery or dine-in. If home delivery is selected, the user provides a delivery address and delivery charges are added; otherwise, the user confirms dine-in. Both paths merge before proceeding to payment.

Next, the user selects an online payment method and the system processes the payment. Another decision checks whether the payment is successful. If successful, the system generates an order confirmation, updates inventory, and notifies the kitchen or dispatches delivery. If payment fails, a failure message is shown. Finally, both outcomes merge and the process ends.

Overall, the diagram clearly shows the flow of activities, decision points, and alternative paths involved in placing an order.

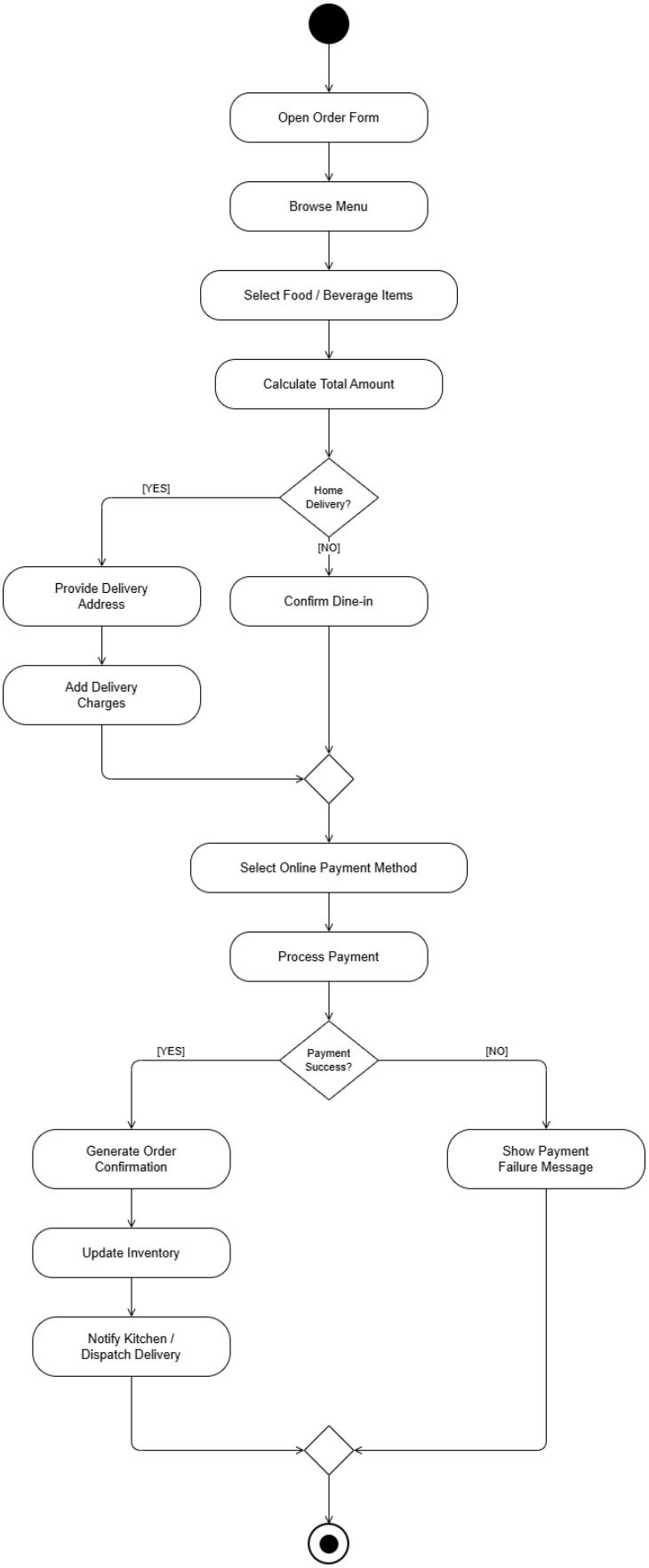

Open Order Form
Browse Menu
Select Food / Beverage Items
Calculate Total Amount
Home Delivery?
[YES]
[NO]
Provide Delivery Address
Confirm Dine-in
Add Delivery Charges
Select Online Payment Method
Process Payment
Payment Success?
[YES]
[NO]
Generate Order Confirmation
Show Payment Failure Message
Update Inventory
Notify Kitchen / Dispatch Delivery

Fig 6: Activity Diagram

5. Table and Class Diagram

5.1 Table

Use Cases	Domain Classes
Login/Registration	1. Account 2. Registration 3. Login 4. Admin 5. Member 6. Visitor 7. Staff
Book Table (Reserve)	1. Reservation 2. Table 3. Member 4. Visitor
Set Beverage Choices	1. BeverageChoice 2. Reservation 3. Member 4. Visitor
Make Advance Payment (Rs 1200)	1. Payment 2. OnlinePayment 3. OfflinePayment 4. Reservation 5. Member
Cancel Reservation	1. Reservation 2. Payment 3. Member 4. Visitor
Place Food/Beverage Order	1. Order

	2. OrderItem 3. MenuItem 4. Payment 5. Member
Rate Experience	1. Rating 2. Feedback 3. Member 4. Visitor
Generate Bill	1. Bill 2. Receipt 3. Payment 4. Admin
Update Menu	1. MenuItem 2. Admin
Generate Financial Report	1. FinancialReport 2. Payment 3. Order 4. Reservation 5. Admin
Post Offers & Discount Schemes	1. Offer 2. Member 3. Admin

Table 3: Table showing all use cases and domain classes

5.2 Class Diagram

Overview

The Analysis Class Diagram for the Gokyo Bistro Management System captures the static structure of the application by identifying all domain classes derived from the use cases, along with their attributes, methods, and the relationships between them. The diagram contains **22 domain classes** organised into five logical layers: an authentication layer (Account, Login, Registration), a user-type layer (Admin, Staff, Member, Visitor), an operational layer for restaurant activities (Reservation, Table, Order, OrderItem, BeverageChoice), a financial layer (Payment, Bill, Receipt, MenuItem), and a specialisation layer (OnlinePayment, OfflinePayment, Offer, FinancialReport, Rating, Feedback).

Class Structure

Each class follows the standard UML three-compartment notation: a header section showing the class name (with the abstract Payment class shown in italics), an attributes section listing data members with their visibility and types, and a methods section listing the operations the class supports. Attributes and methods reflect the requirements described in the coursework specification. For example, the Reservation class holds booking data such as date, time, numGuests, and status, while exposing operations like createReservation() and cancelReservation().

Relationship Summary

The diagram models four kinds of relationships, satisfying and exceeding the coursework requirement of at least two inheritance, two aggregation, and two composition relationships.

Inheritance (5 relationships)

It is represented by a solid line with a hollow triangle pointing to the parent class. Three subclasses inherit from Account: Admin, Staff, and Member, as all three share the common authentication attributes such as userId, username, and password. Two subclasses inherit from the abstract Payment class: OnlinePayment (which adds transactionId and paymentGateway for digital payments) and OfflinePayment (which adds cashierName and paymentTime for in-person payments).

Composition (3 relationships)

It is represented by a solid line with a filled black diamond on the whole side, indicating a strong "owns-a" relationship in which the part cannot exist without the whole. Reservation ◆ BeverageChoice (1 to 0..*)* means beverage selections exist only as part of a reservation. Order ◆ OrderItem *(*1 to 1..) means order items have no meaning outside their parent order. Bill ◆ Receipt (1 to 1) means each bill produces exactly one receipt, and the receipt's lifecycle is tied entirely to its bill.

Aggregation (3 relationships)

It is represented by a solid line with a hollow diamond on the whole side, indicating a weaker "has-a" relationship in which the part can exist independently. Reservation ◇ Table (0..* to 1) means reservations refer to tables, but tables exist independently of any individual reservation. Order ◇ MenuItem (0..* to 1..) means orders reference menu items, but menu items remain in the system regardless of any single order. Admin ◇ Offer (1 to 0..*)* means an admin manages offers, while offers exist as independent entities visible to all users.

Association (12 relationships)

It is represented by plain solid lines with a verb describing the interaction and multiplicity numbers at each end. Key associations include Registration creates Account (1 to 1), Login validates Account (1 to 1), Member makes Reservation (1 to 0..*),* Member places Order (1 to 0..), Member submits Rating (1 to 0..*),* Reservation requires Payment (1 to 1), Order paid by Payment (1 to 1), Bill settles Payment (1 to 1), Admin generates FinancialReport (1 to 0..*),* and Staff handles Order (1 to 0..).

Multiplicities

Multiplicities are placed next to the class they describe, following the UML convention that the number indicates how many instances of that class participate in the relationship. The notations used are 1 (exactly one), 0..* (zero or many), and 1..* (one or many). For example, in the Member makes Reservation association, the 1 sits next to Member and the 0..* sits next to Reservation, meaning each reservation belongs to exactly one member, while a single member may make any number of reservations including none.

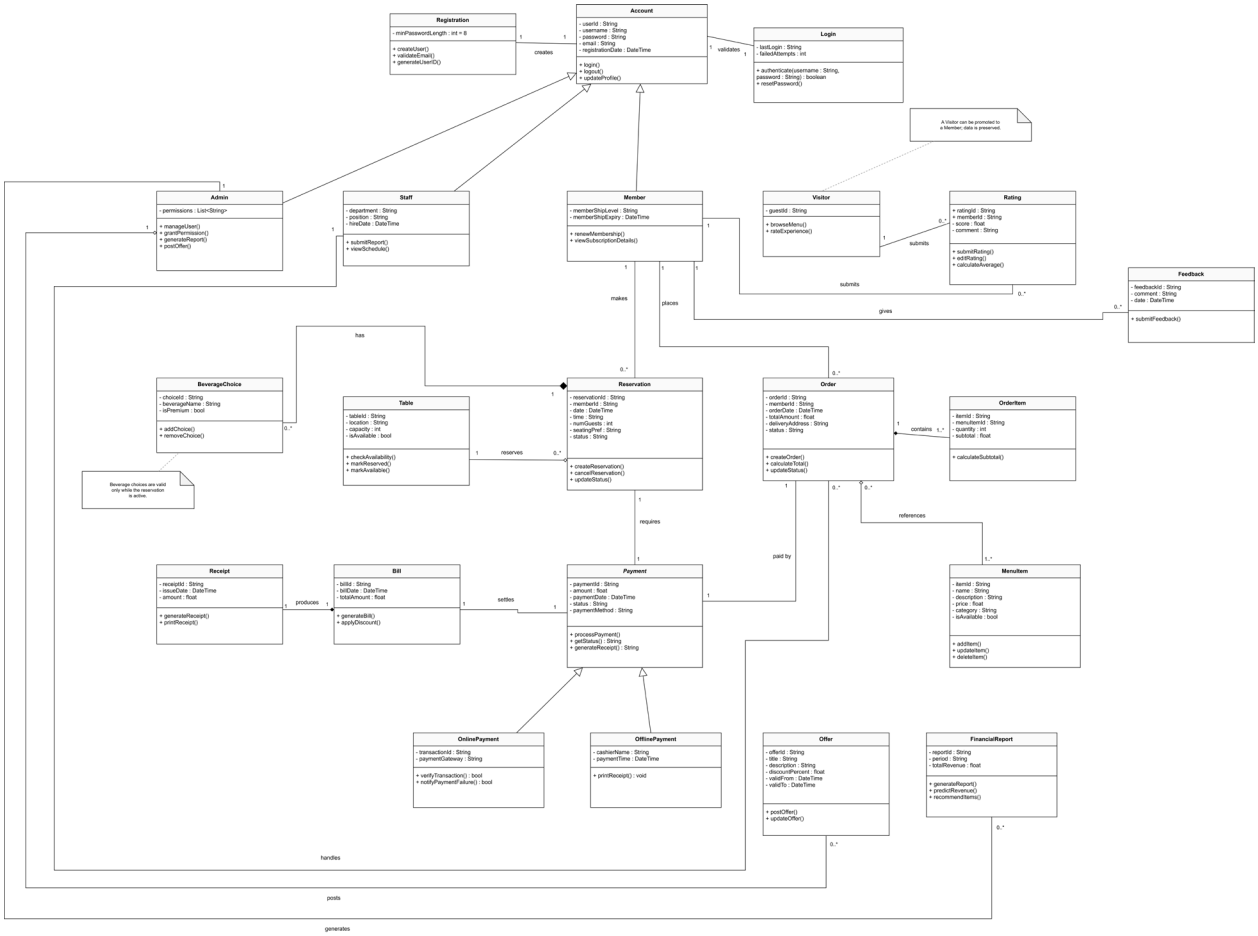


Fig 7: Analysis Class Diagram

6. Further Planning

6.1 Choice of Development Methodology

Now that the analysis and design phase is complete (WBS, use case model, expanded use case descriptions, collaboration diagram, activity diagram, and analysis class diagram), I plan to continue with the **Agile Scrum methodology** to complete the rest of the project. Scrum is a good fit here because the WBS itself was already organised around four sprints, and the client has urgent operational problems (customers being turned away, growing demand for home delivery) that benefit from receiving small working increments early rather than waiting for a single big release. Scrum also handles changing requirements well, which matters because some details. For example, how premium members reserve their wine choices, or how home delivery charges should be calculated which are likely to be refined as the client sees early versions. The team will work in **two-week sprints**, with sprint planning at the start, short daily meetings to coordinate, a sprint review with the client at the end, and a retrospective to reflect on what to improve next time. The product backlog will be drawn directly from the use cases identified earlier.

6.2 Architectural Choice

The system will follow a **three-tier architecture**, which is the standard layered pattern taught in Software Engineering. The **presentation tier** will be the user interface that visitors, members, and admins interact with through their browsers. The **application (logic) tier** will contain the business logic such as booking validation, payment handling, order workflow, report generation and act as the bridge between the user interface and the database. The **data tier** will be the database that stores users, reservations, orders, menu items, payments, bills, offers, and ratings. Separating the system into these three tiers is appropriate for a coursework project because it keeps the design clean and modular: the user interface can be redesigned without touching the business logic, and the database structure can be modified independently. It also matches the analysis class diagram, where boundary, controller, and entity classes naturally map to the three tiers.

6.3 Design Pattern

A few well-known design patterns will be applied to keep the code organised. The **Model-View-Controller (MVC)** pattern will structure the overall application: the model holds the data (such as Reservation, Order, Payment), the view displays it to the user, and the controller handles user input and updates the model. The Factory pattern will be used to create the correct subclass of Payment at runtime which is either OnlinePayment or OfflinePayment depending on the customer's choice, which directly reflects the inheritance shown in the class diagram. The Singleton pattern will be used for the database connection so that only one connection object exists across the system, avoiding wasted resources. These three patterns together cover the main structural concerns of the project.

6.4 Development Plan

For the technology stack, I plan to use familiar and widely-taught tools. The front-end will be built with **HTML, CSS, and JavaScript**, using a framework such as **React** for the user interface. The back-end will be built using **Java** with a web framework, or alternatively **PHP**, depending on team familiarity. **MySQL** will be used as the database. For development tools, the team will use **Visual Studio Code** as the code editor, **Git with GitHub** for version control, and **Postman** for testing the back-end. UI mockups will be drawn in **Draw.io** or **Balsamiq**.

The priority order of features is based on dependency and business value. **Sprint 1** will deliver the login and registration system, since every other feature depends on knowing who the user is. **Sprint 2** will deliver the table booking system (check availability, reserve a table, set beverage choices, advance payment, cancellation), which directly addresses the client's biggest problem of customers being turned away. **Sprint 3** will deliver the food and beverage order placement system with home delivery option and online payment, addressing the second business need. **Sprint 4** will deliver the admin features: update menu, generate bills and receipts, generate financial reports, post offers and discount schemes, and the rating/feedback module. Less critical features will be added later if time permits.

6.5 Testing Plan

Testing will be done throughout the project, not only at the end. Three levels of testing will be carried out. **Unit testing** will be done by the developer for each individual function or class . For example, testing that the reservation creation method correctly rejects an unavailable date. **Integration testing** will check that different modules work together correctly. For example, that a successful payment correctly leads to the reservation being confirmed and the table being marked as reserved. **System testing** will be carried out at the end of each sprint to check that the whole increment works as expected against its requirements. Finally, **user acceptance testing (UAT)** will be done with the Gokyo Bistro staff at the end of each sprint to confirm that the delivered features actually solve their problem. Both **black-box testing** (checking outputs based on inputs without looking at the code) and **white-box testing** (checking the internal logic and code paths) will be applied where appropriate.

6.6 Maintenance Plan

After the system is delivered, maintenance will follow the four standard categories taught in Software Engineering. Corrective maintenance will fix any bugs that are discovered after release, based on reports from the staff or customers. Adaptive maintenance will keep the system working when its environment changes. For example, if the payment gateway updates its API, or if the operating system or browser changes. Perfective maintenance will introduce improvements requested by the client over time, such as new offer types, extra report fields, or small interface improvements based on user feedback. Preventive maintenance will involve regular database backups, applying security updates, and tidying up the code to prevent problems from building up. Clear documentation both technical (for future developers) and user manuals (for the bistro staff) will be prepared so that the system can be supported even if the original team is no longer available.

6.7 Conclusion

By following the Scrum methodology with a three-tier architecture, applying standard design patterns like MVC, Factory, and Singleton, planning sprints in priority order, testing throughout development, and maintaining the system properly after delivery, the Gokyo Bistro Management System can be completed successfully and continue to serve the client's needs over time. This plan covers all the standard phases of the software development life cycle and builds directly on the analysis and design work already produced.

7. Design Prototypes

16 different prototypes along with their brief introduction are attached here below. These prototypes were drawn on a software called draw.io.

Prototype 1: Home Page

This page introduces Gokyo Bistro and provides quick navigation to important services such as booking a table, viewing the menu, ordering online, and login/registration.

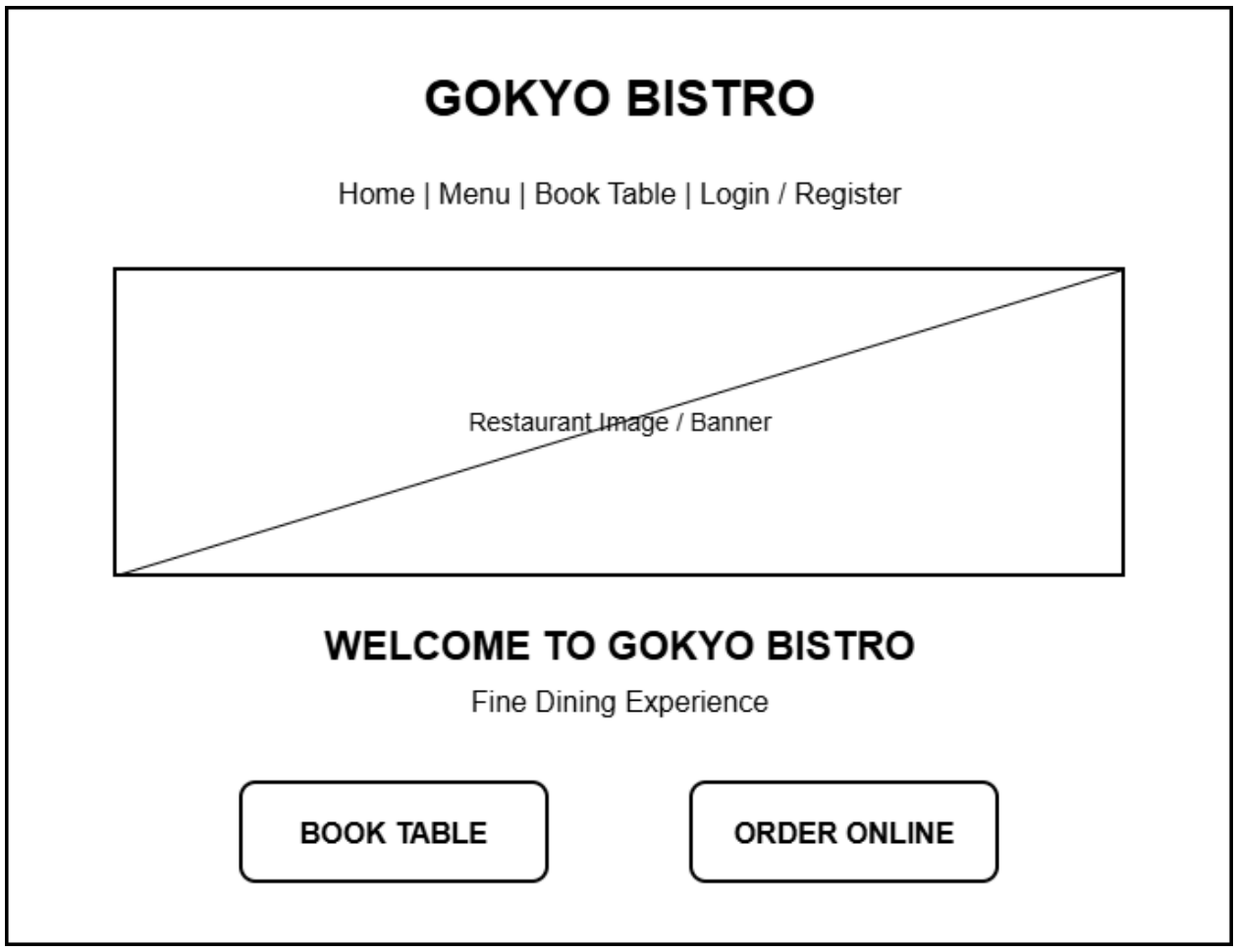


Fig 8: Prototype 1: Home Page

Prototype 2: Login Page

This page allows registered members and admin users to log in using their email/username and password.

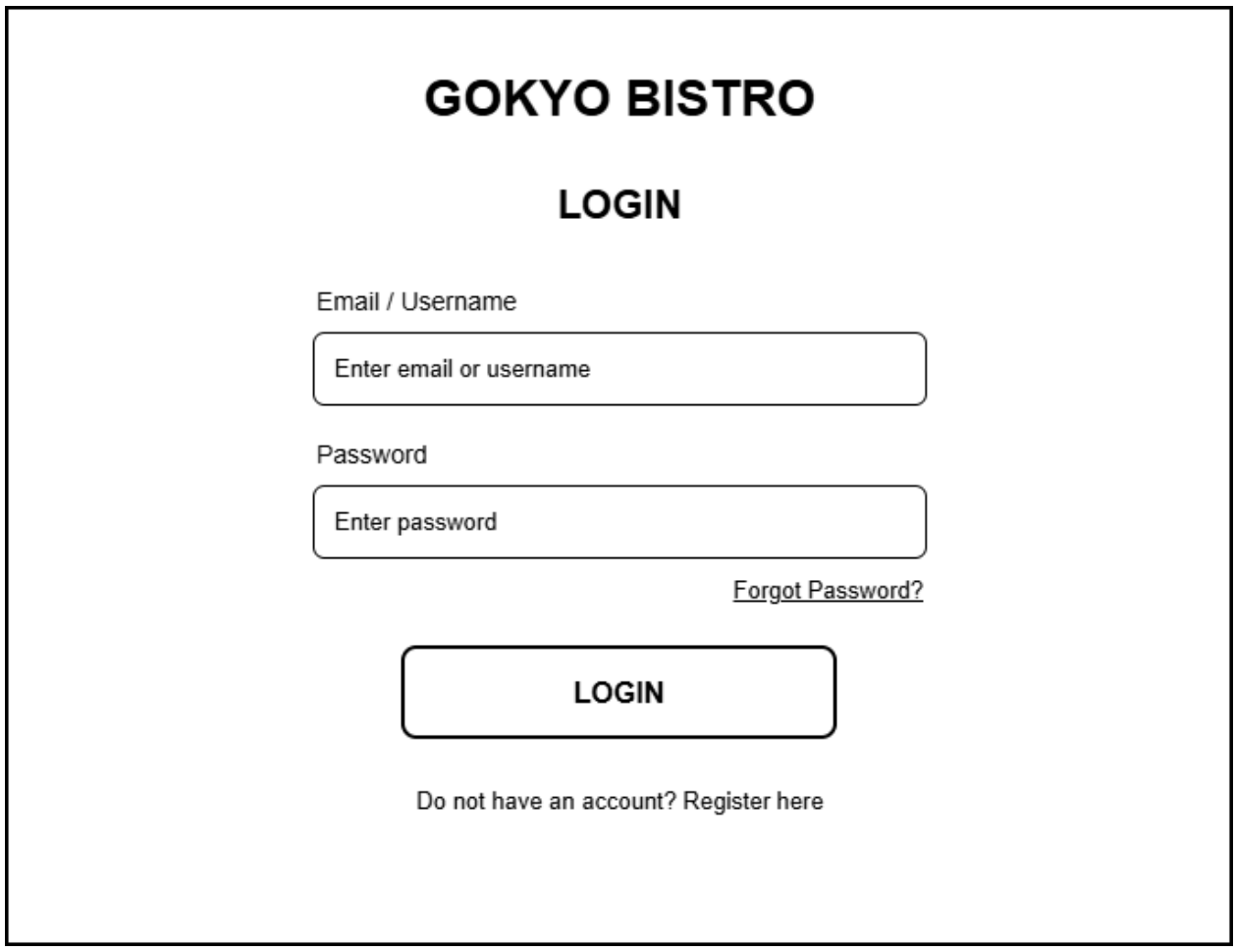


Fig 9: Prototype 2: Login Page

Prototype 3: Registration Page

This page allows new visitors to create an account by entering personal details such as name, email, phone number, and password.

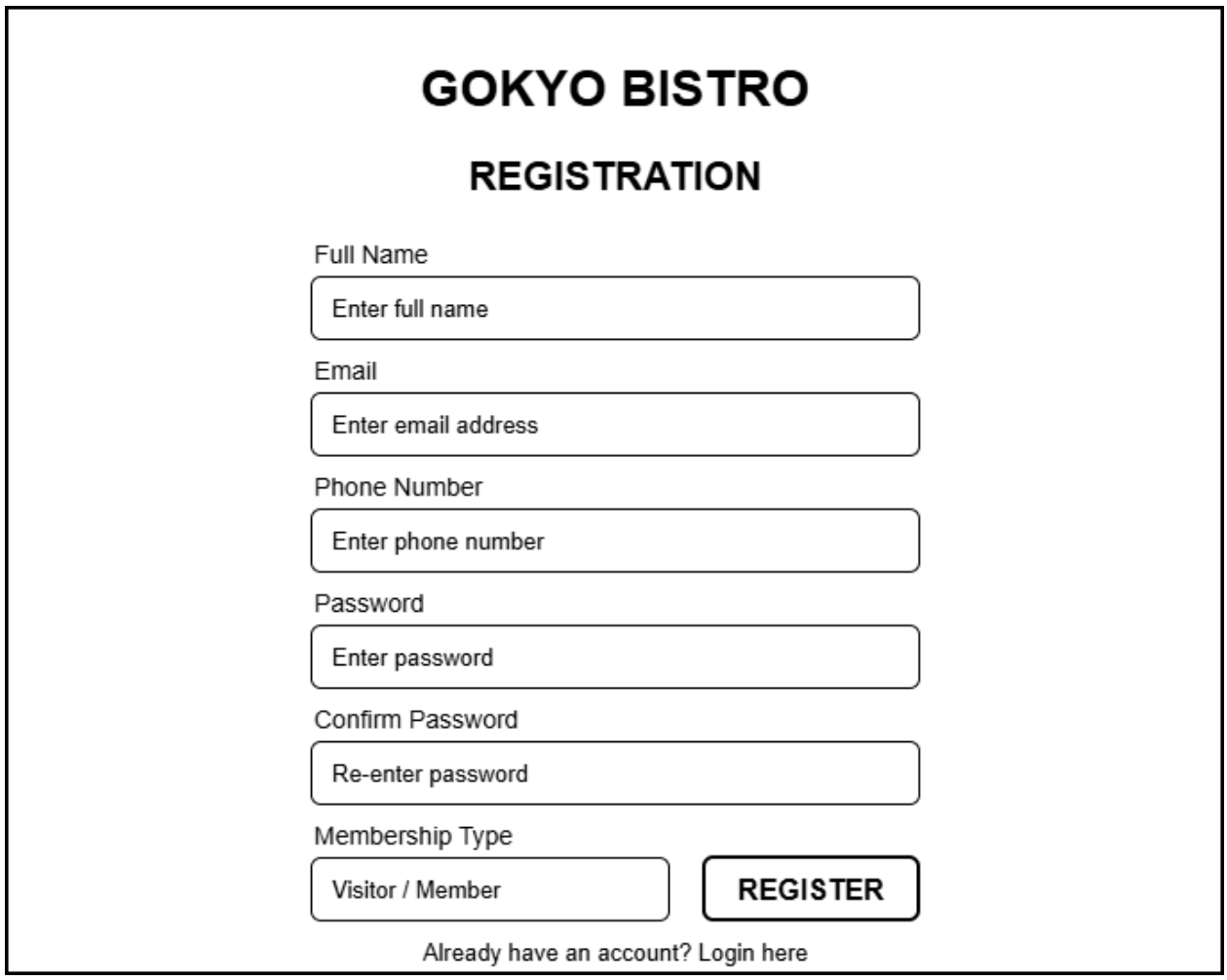


Fig 10: Prototype 3: Registration Page

Prototype 4: Reset Password Page

This page helps users reset their password if they forget their login credentials.



Fig 11: Prototype 4: Reset Password Page

Prototype 5: User Dashboard

This page provides visitors/members with quick access to booking tables, ordering food, viewing reservations, and rating their experience.

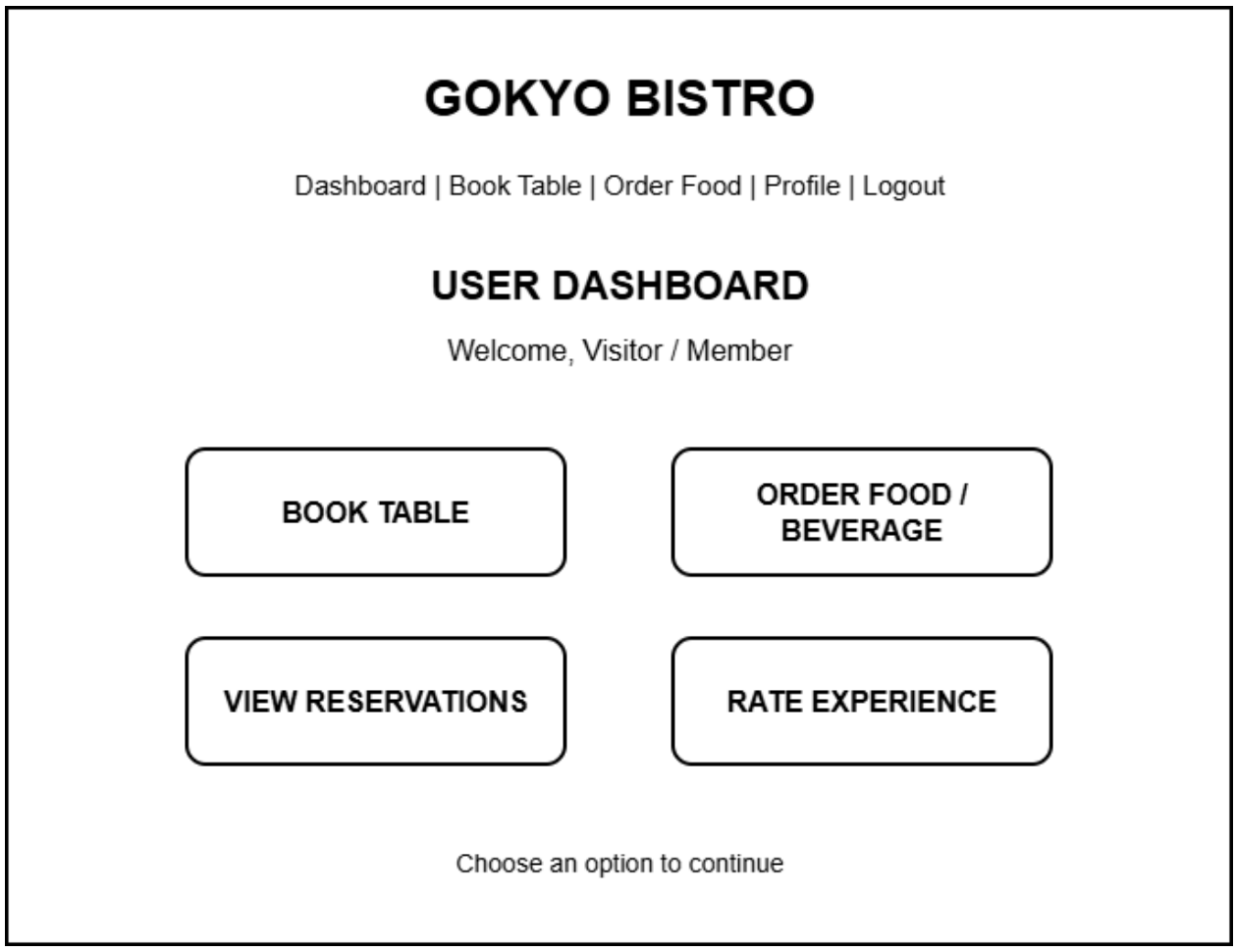


Fig 12: Prototype 5: User Dashboard

Prototype 6: Check Table Availability

This page allows users to check table availability by selecting date, time, and number of guests.



Fig 13: Prototype 6: Check Table Availability

Prototype 7: Book Table Page

This page allows users to reserve an available table by confirming booking details, seating preference, and special requests.

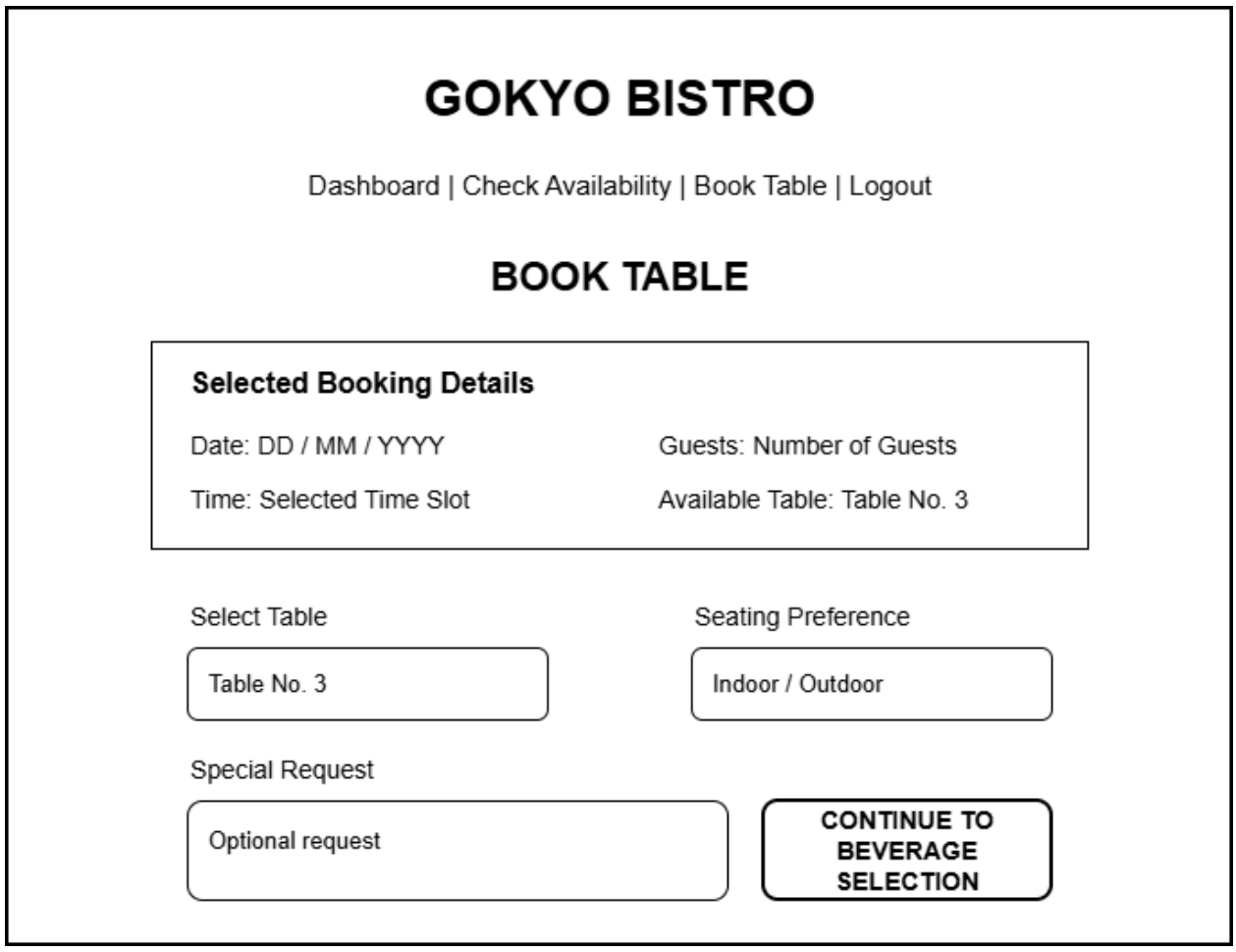


Fig 14: Prototype 7: Book Table Page

Prototype 8: Beverage Selection Page

This page allows users to select expected beverage choices, including premium wine or whiskey options for premium members.

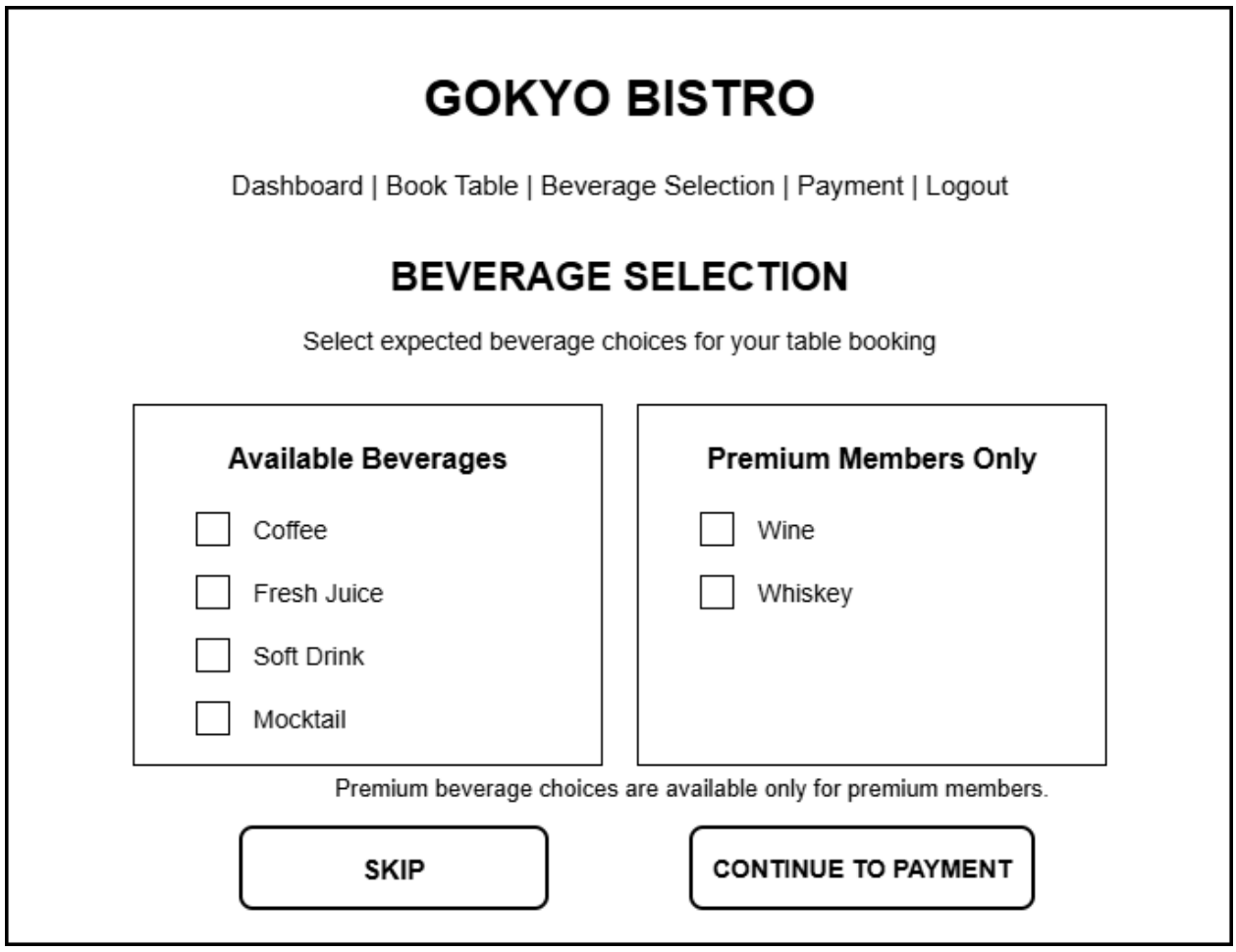


Fig 15: Prototype 8: Beverage Selection Page

Prototype 9: Advance Payment Page

This page allows users to pay the required Rs 1200 advance amount to confirm the table reservation.

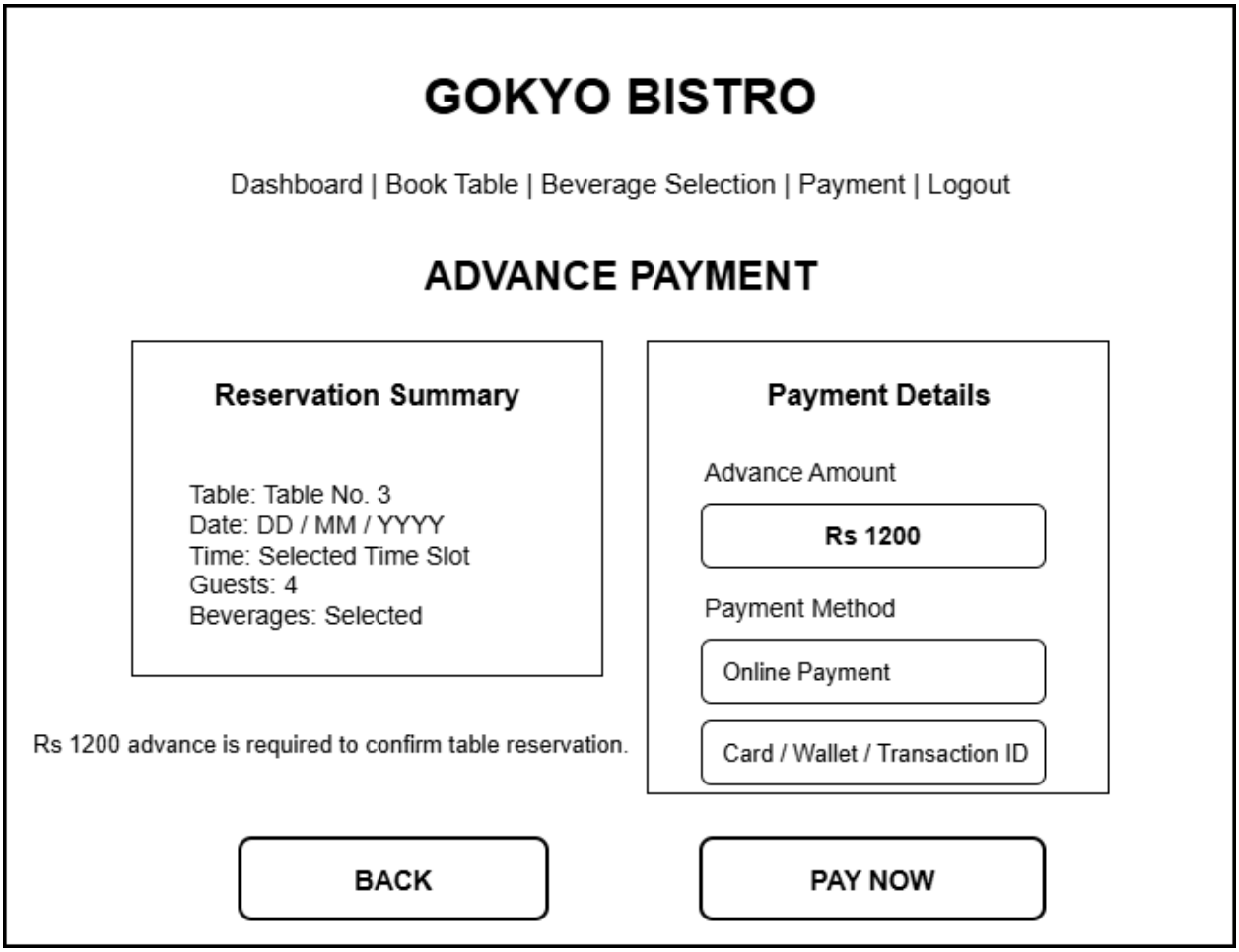


Fig 16: Prototype 9: Advance Payment Page

Prototype 10: Reservation Confirmation Page

This page displays the confirmed reservation details such as booking ID, table number, date, time, guest count, and payment status.



Fig 17: Prototype 10: Reservation Confirmation Page

Prototype 11: Cancel Reservation Page

This page allows users to cancel a reservation and view the applicable refund or cancellation fee details.

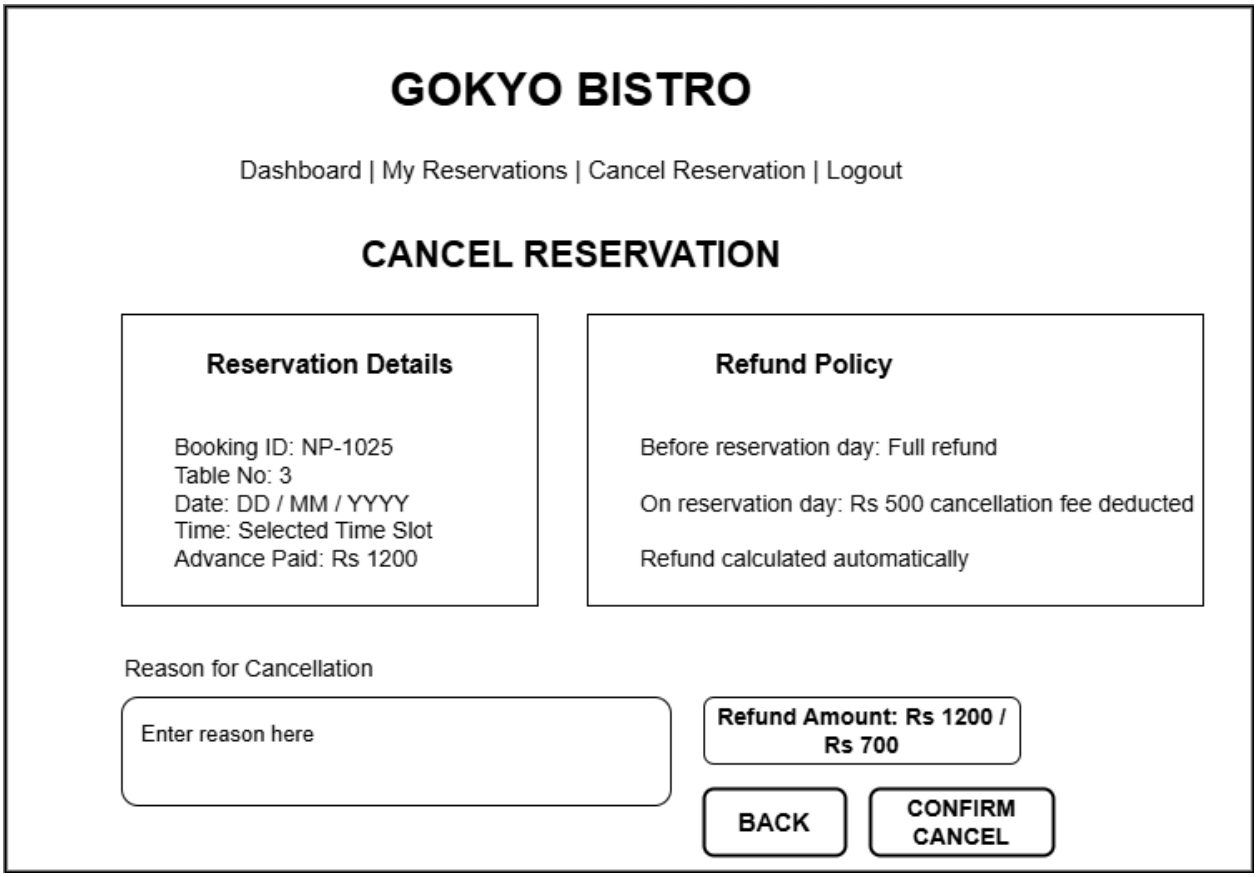


Fig 18: Prototype 11: Cancel Reservation Page

Prototype 12: Order Placement Page

This page allows members to browse menu items, add food or beverages to the cart, and proceed with ordering.

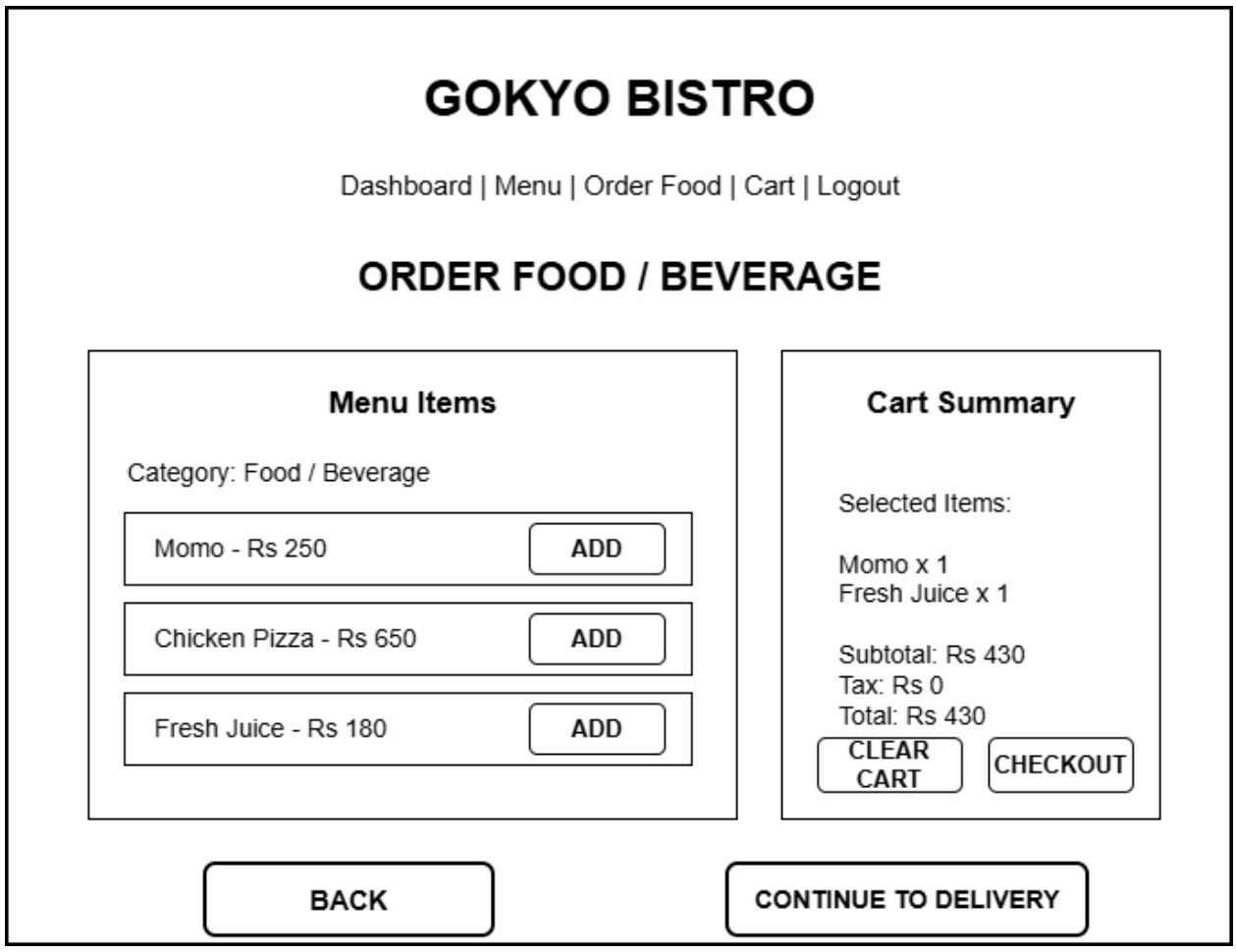


Fig 19: Prototype 12: Order Placement Page

Prototype 13: Delivery Option Page

This page allows members to choose between dine-in/pickup and home delivery, including entering a delivery address if required.



Fig 20: Prototype 13: Delivery Option Page

Prototype 14: Bill & Receipt Page

This page displays billing details, ordered items, total amount, payment status, and provides an option to download the receipt.



Fig 21: Prototype 14: Bill & Receipt Page

Prototype 15: Admin Dashboard Page

This page provides admin access to major management functions such as updating menu items, generating bills, reports, revenue prediction, offers, and feedback management.

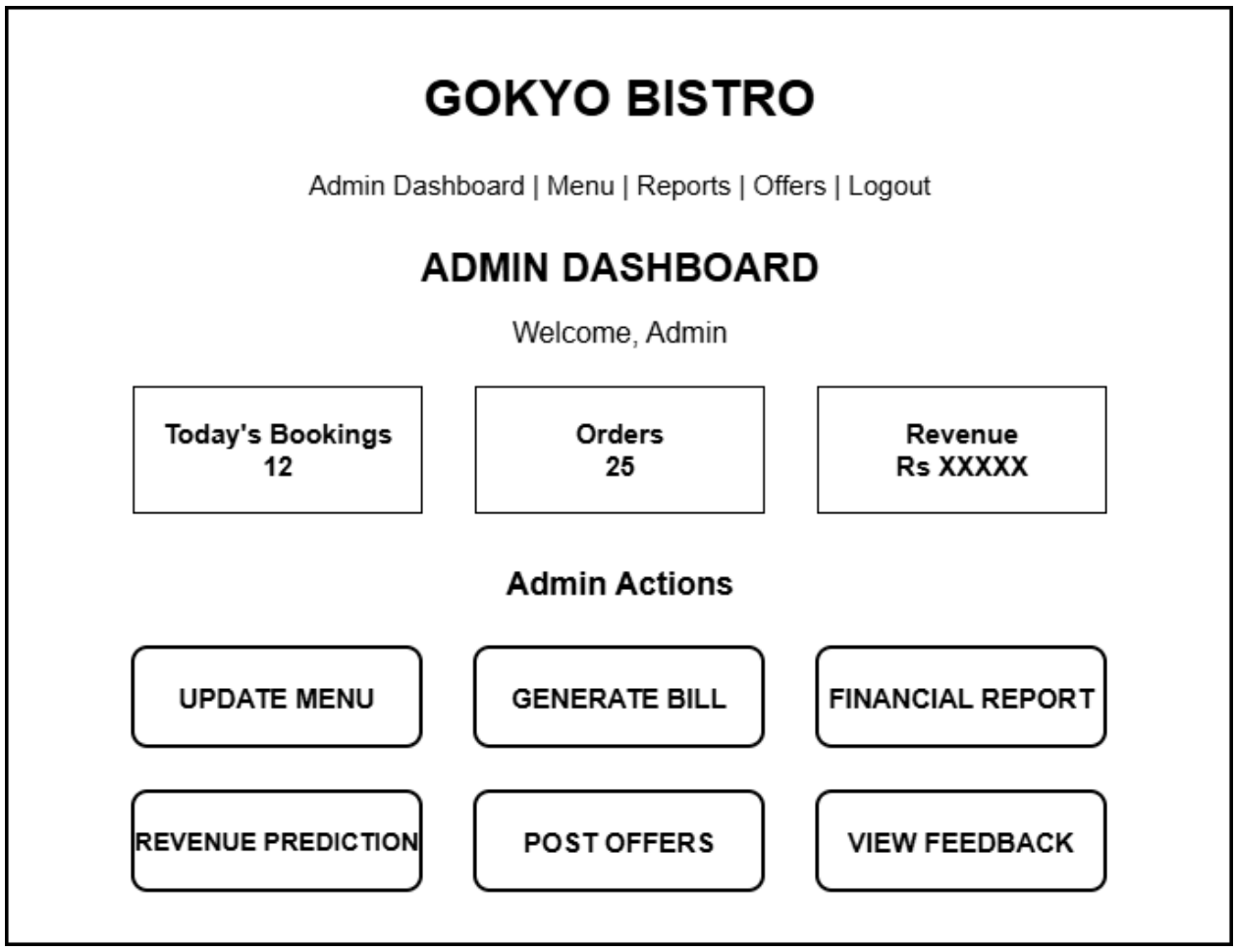


Fig 22: Prototype 15: Admin Dashboard Page

Prototype 16: Rate Experience / Feedback Page

This page allows visitors or members to rate food quality, staff behaviour, ambience, and overall experience, along with written feedback.

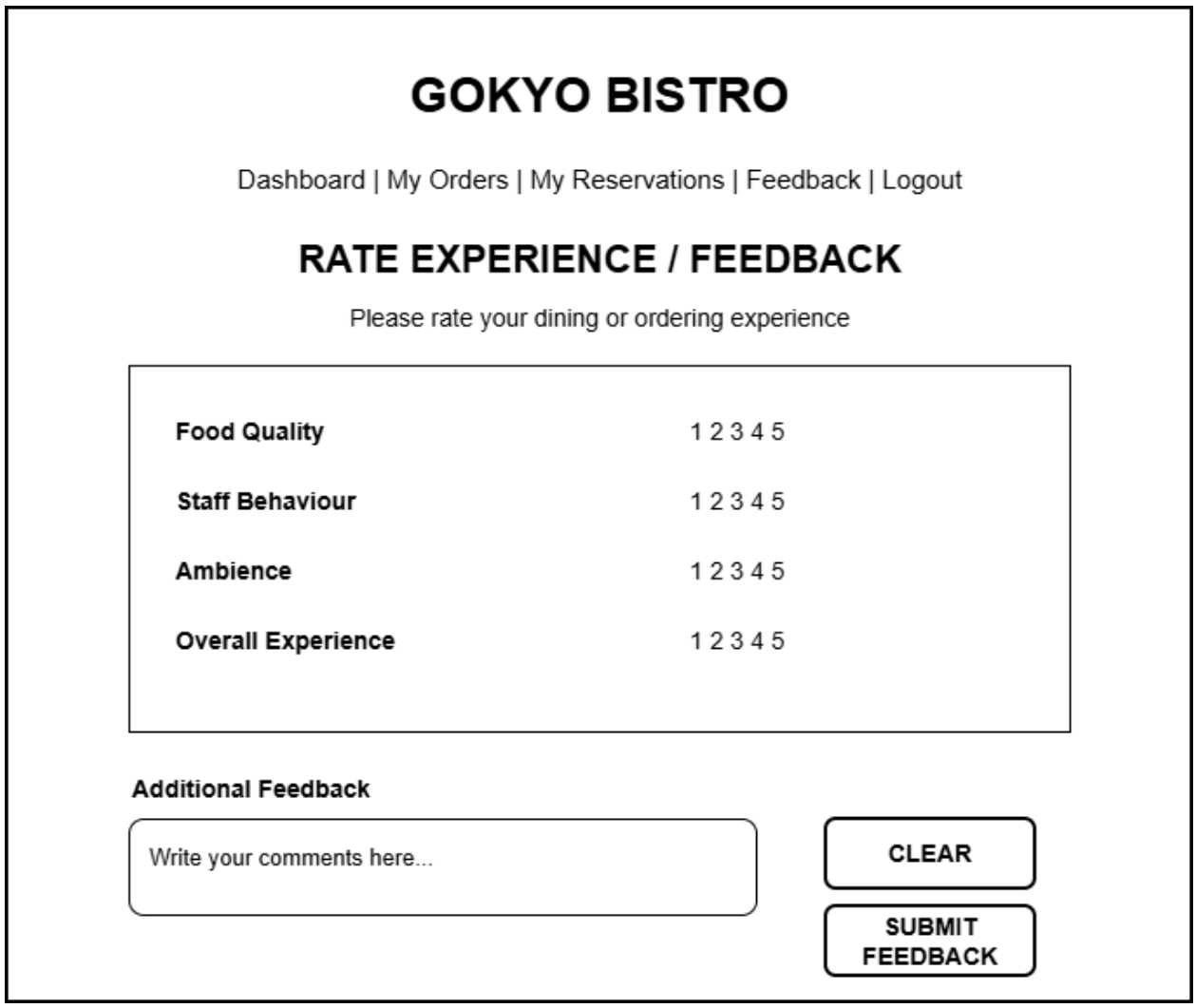


Fig 23: Prototype 16: Rate Experience/Feedback Page